

资源受限铁道知识图片描述软件 软件设计说明书

软件名称：资源受限铁道知识图片描述软件 V1.0

版本号：V1.0

文档类型：软件设计说明书

目录

1 引言

资源受限铁道知识图片描述软件面向实际业务中的资料处理、过程控制、结果记录和运行维护场景，围绕当前仓库已经实现或明确配置的入口组织功能边界。系统以Python、PostgreSQL作为主要技术支撑，针对训练、评分、评测或结果记录流程、图像/视频输入、帧处理或识别结果管理、三维模型、地理场景或 WebGL 交互建立可追踪的处理链路。使用者从项目定义的入口提交输入或选择任务，系统完成参数校验、业务处理、状态更新和结果反馈；在数据需要持久化时，通过项目已有的数据层或本地文件结构保存关键对象，避免把临时过程误写成登记功能。

在知识库、向量检索或增强式问答链路、结果导出、格式转换或报告生成、规则判断、风险识别或异常分支方面，软件根据真实代码中的对象、配置键、接口或命令进行组织。输入可以来自页面表单、图片视频、文本语料、训练配置、模型文件或命令行参数，输出可以是识别结果、检索结果、练习记录、评分报告、图形场景、导出文件或测试日志。对于不具备完整运行入口的仓库，文档只描述已经存在的脚本、模块和配置，并将尚未验证的页面或外部服务列为受阻项。

软件提供与项目主题相匹配的异常处理和可恢复路径，例如参数缺失、文件格式不符、模型或数据库不可用、权限不足、外部服务不可达、任务失败或结果为空等情况。部署时优先采用仓库已有的安装、构建、启动和测试命令，在 Windows 与 Linux 环境中通过路径 API、环境变量和 UTF-8 文本读取保持配置边界清晰；服务器地址、账号、令牌、密码和本机路径在软著材料中统一脱敏。

材料范围包括软件设计说明书、软件源代码和用户操作手册。设计说明书只说明需求概要、架构、业务逻辑、数据、接口、部署、安全和测试设计，并使用由代码推导出的项目专属设计图；源代码材料以真实自研源码为主体；用户操作手册只记录真实可达的页面、命令、服务或设备流程，运行截图仅放入用户操作手册，不与设计说明书混用。

项目说明中的已核验摘要：受限资源下的铁道知识图片描述 <p align="center"> 这是一个用于论文实验的代码框架，目标是支撑： - 铁路专用图文数据集构建 - 铁路知识库/术语表约束 - 轻量级图像描述模型训练 - 蒸馏、量化、导出等资源受限部署实验 - 通用 caption 指标与铁路领域指标评估 数据格式 训练、验证、测试集使用 JSONL 清单，每行一条样本： 字段说明： - image：图片路径，可为相对仓库根目录路径或绝对路径 - caption：人工标注或知识驱动生成的描述 - equipment：图片中出现的铁路设备/部件 - faults：缺陷或异常术语，没有则为空数组 - risks：风险语义标签，例如 异物侵限风险、扣件松动风险 - split：train、val 或 test 快速开始 目录结构 论文实验建议 核心对比组建议至少包含： 1. 通用 caption 模型直接微调 2. 加铁路术语约束 3. 加知识增强提示或检索 4. 小模型蒸馏 5. 量化/导出后的边缘部署结果 领域指标建议报告： - 设备术语召回率 - 故障术语 F1 - 风险描述准确率 - 铁路术语幻觉率 - 参数量、显存、延迟、吞吐量 <!-- codex-runtime-notes:start -- Runtime Ports And Database Configuration

1.1 软件特色摘要

1. 训练、评分、评测或结果记录流程，对应代码标识符：src/railcap/__init__.py。
2. 图像/视频输入、帧处理或识别结果管理，对应代码标识符：src/railcap/benchmark.py。
3. 三维模型、地理场景或 WebGL 交互，对应代码标识符：src/railcap/config.py。
4. 知识库、向量检索或增强式问答链路，对应代码标识符：src/railcap/evaluate.py。
5. 结果导出、格式转换或报告生成，对应代码标识符：src/railcap/knowledge/__init__.py。

2 需求概要分析

2.1 需求背景

资源受限铁道知识图片描述软件的建设背景来自项目仓库中已经形成的业务描述、模块代码、配置文件和脚本。目标用户或使用方通过真实入口提交业务输入、执行任务或查看处理结果，软件需要在本地开发环境和可配置的部署环境中稳定完成数据处理、状态反馈和结果留存。对于当前仓库尚未实现的外部服务或页面，不在本说明书中虚构其存在。

2.2 建设目标与目标用户

建设目标是把训练、评分、评测或结果记录流程、图像/视频输入、帧处理或识别结果管理、三维模型、地理场景或 WebGL 交互、知识库、向量检索或增强式问答链路组织为可验证的软件流程。目标用户依据项目实际形态分为页面操作人员、任务/实验执行人员、运维人员或开发测试人员；具体权限以代码中的路由、命令入口、配置和服务边界为准。

2.3 核心业务需求

核心业务需求包括输入校验、任务或页面入口、处理过程、状态与异常、结果输出、数据保存和必要的日志记录。每项需求均应回到真实代码路径验证，关键标识符包括：`src/railcap/__init__.py`、`src/railcap/benchmark.py`、`src/railcap/config.py`、`src/railcap/evaluate.py`、`src/railcap/knowledge/__init__.py`、`src/railcap/knowledge/constraints.py`。

2.4 数据与接口需求

输入输出数据可能由文本、结构化记录、图片视频、模型参数、配置项或命令行参数构成。系统应对缺少字段、格式不符、文件不存在、依赖服务不可用和结果为空等情况给出明确错误路径；通信地址、数据库连接和密钥在材料中使用脱敏示例。

2.5 运行、安全与测试概要要求

运行环境按当前平台预检记录执行，文本材料使用 UTF-8。安全要求包括敏感配置不进入文档正文、输入边界校验、权限或会话检查、异常日志和结果隔离。测试要求覆盖构建/启动、核心功能、接口或命令、异常输入、权限边界及结果导出。

3 系统概述

3.1 系统边界

系统边界由Python、PostgreSQL和项目源码目录中的实际入口组成，外部模型、数据库、浏览器、设备、第三方服务或数据集仅在代码或配置中明确引用时纳入边界。软件运行截图不属于本章素材。

3.2 功能模块

序号	模块/入口	核验依据	主要职责
1	训练、评分、评测或结果记录流程	<code>src/railcap/__init__.py</code>	接收输入、执行当前项目对应处理并输出状态或结果
2	图像/视频输入、帧处理或识别结果管理	<code>src/railcap/benchmark.py</code>	接收输入、执行当前项目对应处理并输出状态或结果

序号	模块/入口	核验依据	主要职责
3	三维模型、地理场景或 WebGL 交互	src/railcap/config.py	接收输入、执行当前项目对应处理并输出状态或结果
4	知识库、向量检索或增强式问答链路	src/railcap/evaluate.py	接收输入、执行当前项目对应处理并输出状态或结果
5	结果导出、格式转换或报告生成	src/railcap/knowledge/__init__.py	接收输入、执行当前项目对应处理并输出状态或结果
6	规则判断、风险识别或异常分支	src/railcap/knowledge/constraints.py	接收输入、执行当前项目对应处理并输出状态或结果

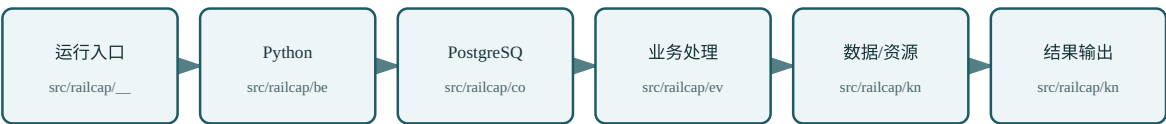
3.3 运行关系与技术范围

软件按项目真实结构运行于本地进程、浏览器、移动端、容器、GPU 环境或命令行任务中。技术范围包括 Python、PostgreSQL；不把未在代码中出现的菜单、角色、数据库或外部接口写入设计范围。

4 总体设计

4.1 总体架构

资源受限铁道知识图片描述软件 - 总体架构



标识符：src/railcap/___init__.py src/railcap/benchmark.py src/railcap/config.py src/railcap/evaluate.py src/railcap/knowledge/___init__.py

图 4-1 总体架构图。图中以真实技术栈和项目标识符表示入口、处理层、数据/资源和结果边界。

4.2 部署设计

资源受限铁道知识图片描述软件 - 部署关系

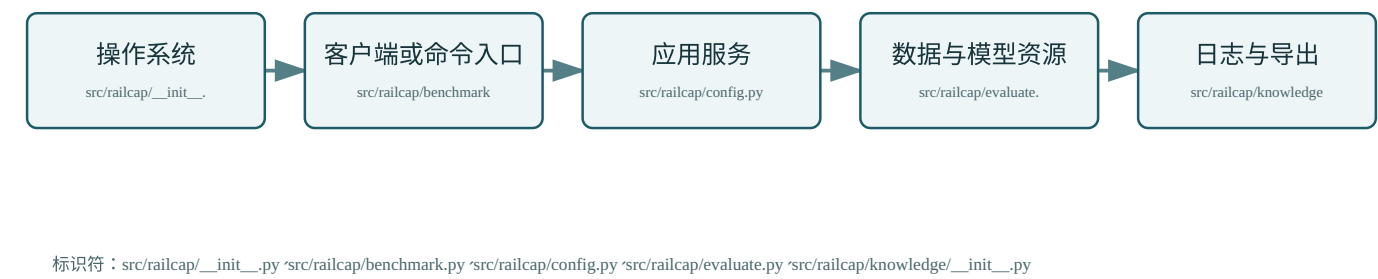


图 4-2 部署关系图。部署地址、端口和密钥只在运行环境中配置，材料中使用脱敏值。

4.3 主要技术描述

前端或交互层负责真实入口的输入采集、状态呈现和结果操作；服务或任务层负责训练、评分、评测或结果记录流程、图像/视频输入、帧处理或识别结果管理、三维模型、地理场景或 WebGL 交互；数据层负责项目实际使用的文件、数据库、模型或实验记录；构建和测试层使用Python、PostgreSQL对应的仓库脚本。

5 详细设计

5.1 核心处理流程

资源受限铁道知识图片描述软件 - 关键处理流程

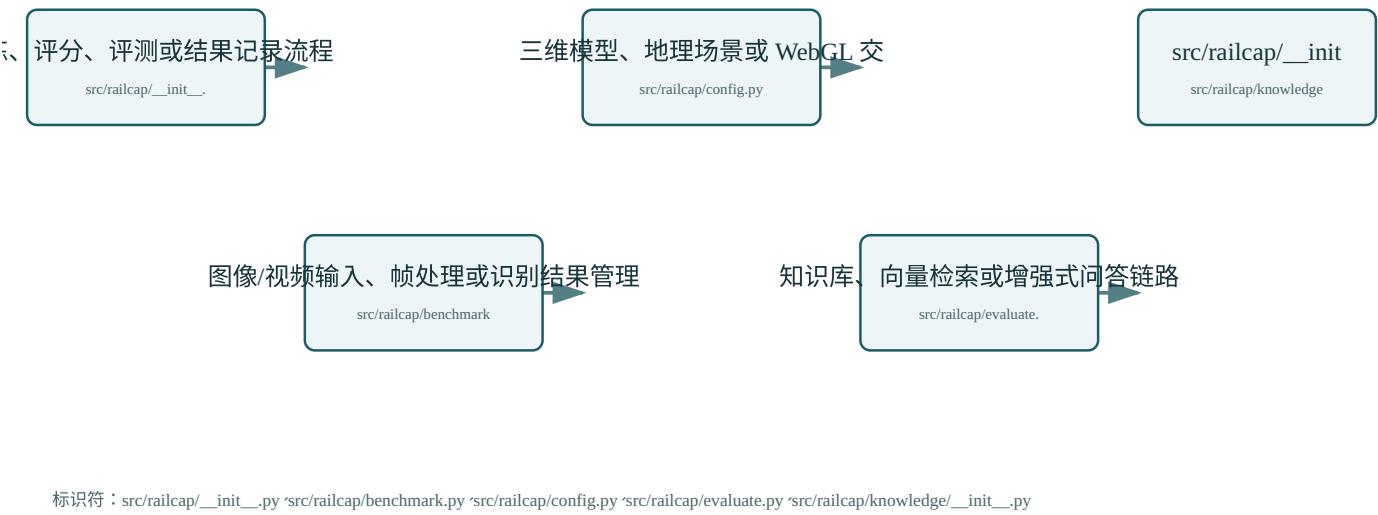


图 5-1 核心处理流程图。处理过程包含输入、项目专属机制、结果与异常分支。

5.2 训练、评分、评测或结果记录流程

该设计单元以 src/railcap/__init__.py 为主要代码证据，定义输入、处理规则、输出状态和异常处理。当输入不满足约束时，流程转入错误反馈或日志记录，不继续写入不完整结果。

5.3 图像/视频输入、帧处理或识别结果管理

该设计单元以 src/railcap/benchmark.py 为主要代码证据，定义输入、处理规则、输出状态和异常处理。处理完成后保存或返回当前项目实际使用的结果对象，供后续页面、脚本、服务或导出功能读取。

5.4 三维模型、地理场景或 WebGL 交互

该设计单元以 src/railcap/config.py 为主要代码证据，定义输入、处理规则、输出状态和异常处理。当输入不满足约束时，流程转入错误反馈或日志记录，不继续写入不完整结果。

5.5 知识库、向量检索或增强式问答链路

该设计单元以 src/railcap/evaluate.py 为主要代码证据，定义输入、处理规则、输出状态和异常处理。处理完成后保存或返回当前项目实际使用的结果对象，供后续页面、脚本、服务或导出功能读取。

5.6 结果导出、格式转换或报告生成

该设计单元以 src/railcap/knowledge/__init__.py 为主要代码证据，定义输入、处理规则、输出状态和异常处理。当输入不满足约束时，流程转入错误反馈或日志记录，不继续写入不完整结果。

5.7 规则判断、风险识别或异常分支

该设计单元以 src/railcap/knowledge/constraints.py 为主要代码证据，定义输入、处理规则、输出状态和异常处理。处理完成后保存或返回当前项目实际使用的结果对象，供后续页面、脚本、服务或导出功能读取。

6 数据设计

6.1 数据对象与字段

数据对象	来源	关键字段/标识	状态或约束
src/railcap/__init__.py	代码、配置或运行输入	项目标识 训练、评分、评测或结果记录流程	字段校验、缺省值和异常路径以实现为准
src/railcap/benchmark.py	代码、配置或运行输入	项目标识 图像/视频输入、帧处理或识别结果管理	字段校验、缺省值和异常路径以实现为准
src/railcap/config.py	代码、配置或运行输入	项目标识 三维模型、地理场景或 WebGL 交互	字段校验、缺省值和异常路径以实现为准
src/railcap/evaluate.py	代码、配置或运行输入	项目标识 知识库、向量检索或增强式问答链路	字段校验、缺省值和异常路径以实现为准
src/railcap/knowledge/__init__.py	代码、配置或运行输入	项目标识 结果导出、格式转换或报告生成	字段校验、缺省值和异常路径以实现为准
src/railcap/knowledge/constraints.py	代码、配置或运行输入	项目标识 规则判断、风险识别或异常分支	字段校验、缺省值和异常路径以实现为准

6.2 数据流与持久化

输入经过读取、校验、处理和结果写入或返回；存在数据库时以迁移、模型和查询代码为准，存在模型或文件资源时以配置路径和加载函数为准。敏感数据不直接写入软著材料。

7 接口设计

7.1 关键接口与调用约定

接口/入口	代码证据	调用方	参数/输入	返回/结果
接口/入口 1	src/railcap/__init__.py	项目调用方 或命令入口	参数按真实配置 或函数签名传入	结果对象或状态码；网络地址已脱敏
接口/入口 2	src/railcap/benchmark.py	项目调用方 或命令入口	参数按真实配置 或函数签名传入	结果对象或状态码；网络地址已脱敏
接口/入口 3	src/railcap/config.py	项目调用方 或命令入口	参数按真实配置 或函数签名传入	结果对象或状态码；网络地址已脱敏
接口/入口 4	src/railcap/evaluate.py	项目调用方 或命令入口	参数按真实配置 或函数签名传入	结果对象或状态码；网络地址已脱敏
接口/入口 5	src/railcap/knowledge/__init__.py	项目调用方 或命令入口	参数按真实配置 或函数签名传入	结果对象或状态码；网络地址已脱敏
接口/入口 6	src/railcap/knowledge/constraints.py	项目调用方 或命令入口	参数按真实配置 或函数签名传入	结果对象或状态码；网络地址已脱敏

7.2 脱敏示例

```
{ "input": "***", "page": 1, "pageSize": 10, "authorization": "Bearer ***", "result": "按当前项目实际对象返回" }
```

示例只表达字段关系，不披露真实地址、账号、令牌、密码、IP 或数据库连接信息。

8 运行与部署设计

8.1 环境准备

当前执行环境为 Linux Ubuntu 24.04.4 LTS，使用 UTF-8、x64 架构和 Python、PostgreSQL。Windows 运行时应使用平台无关路径 API、环境变量和对应 shell 命令；本批次仅在 Linux 机器完成材料生成，Windows 端分支需按仓库实际脚本复核。

8.2 启动、构建与检查

优先执行仓库已有的依赖安装、构建、启动和测试脚本。若仓库仅包含研究代码、占位 README 或缺少入口，则将运行状态记录为“受阻/待作者确认”，不把静态源码当成真实运行截图。

8.3 端口、资源和失败处理

端口、数据库、模型、浏览器、GPU、摄像头或外部服务均以当前项目配置为准；材料不写入真实连接值。启动检查至少包括命令返回码、进程状态、健康接口或页面入口、文件读写和结果输出。

9 安全与权限设计

安全设计覆盖配置脱敏、路径边界、输入校验、会话或角色校验、文件类型检查、异常日志、结果隔离和依赖服务失败处理。代码证据包括 src/railcap/__init__.py、src/railcap/benchmark.py、src/railcap/config.py、src/railcap/evaluate.py、src/railcap/knowledge/__init__.py；缺少认证实现的项目不虚构认证功能，而标记为适用性待确认。

10 测试与验证

测试类别	测试对象	步骤	预期结果
构建/启动测试	src/railcap/__init__.py	执行仓库已有构建或启动命令	命令成功或明确记录受阻原因
功能/处理测试	训练、评分、评测或结果记录流程	输入有效样本并执行主流程	得到项目实际定义的结果对象或状态
异常测试	图像/视频输入、帧处理或识别结果管理	输入缺失、格式错误或依赖不可用	返回错误、日志或受控失败
安全测试	配置与边界	检查敏感配置、路径、权限和脱敏	无真实密钥、令牌、账号或地址泄露
接口/回归测试	src/railcap/__init__.py、src/railcap/benchmark.py、src/railcap/config.py	重复执行关键入口并检查输出	结果结构稳定且无明显重叠/乱码

11 结论

本说明书依据当前项目真实代码、配置、README 和可用运行资产整理。设计说明书只保留逻辑与系统设计内容；页面运行截图、命令操作截图和设备截图归入用户操作手册，源码正文独立归入软件源代码材料。

5A 逐机制详细设计、公共报文与测试记录

本节把前述真实机制逐项展开。每个设计单元均绑定代码路径或配置标识，描述正常链路、状态变化、公共报文、异常分支和测试验收条件。<div class="page-break">

5.1 1.1 训练、评分、评测或结果记录流程——业务职责与边界

本单元以 src/railcap/__init__.py 为核验依据，围绕“训练、评分、评测或结果记录流程”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 1

训练、评分、评测或结果记录流程



图示说明：从 src/railcap/__init__.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-1 训练、评分、评测或结果记录流程详细处理图。图中从 src/railcap/__init__.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/__init__.py
处理过程	按状态推进，不跨越失败状态	训练、评分、评测或结果记录流程
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“训练、评分、评测或结果记录流程”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-1.svg，图示以 src/railcap/__init__.py 为项目专属标识。

5.2 1.2 训练、评分、评测或结果记录流程——输入输出与数据约束

本单元以 src/railcap/__init__.py 为核验依据，围绕“训练、评分、评测或结果记录流程”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/__init__.py
处理过程	按状态推进，不跨越失败状态	训练、评分、评测或结果记录流程
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“训练、评分、评测或结果记录流程”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.3 1.3 训练、评分、评测或结果记录流程——状态转换与处理规则

本单元以 src/railcap/__init__.py 为核验依据，围绕“训练、评分、评测或结果记录流程”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/__init__.py
处理过程	按状态推进，不跨越失败状态	训练、评分、评测或结果记录流程
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“训练、评分、评测或结果记录流程”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.4 1.4 训练、评分、评测或结果记录流程——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":" 训 练 、 评 分 、 评 测 或 结 果 记 录 流 程 ","payload":"*","page":1,"pageSize":10}; 成 功 响 应 :
{"code":0,"message":"ok","status":"completed","data":" 按 项 目 实 际 对 象 返 回 "}; 失 败 响 应 :
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/__init__.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/__init__.py
处理过程	按状态推进，不跨越失败状态	训练、评分、评测或结果记录流程
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“训练、评分、评测或结果记录流程”详细处理图一致，图中标示的输入、处理、
结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.5 1.5 训练、评分、评测或结果记录流程——异常分支与安全控制

本单元以 src/railcap/__init__.py 为核验依据，围绕“训练、评分、评测或结果记录流程”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/__init__.py
处理过程	按状态推进，不跨越失败状态	训练、评分、评测或结果记录流程
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“训练、评分、评测或结果记录流程”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.6 1.6 训练、评分、评测或结果记录流程——测试设计与验收准则

本单元以 src/railcap/__init__.py 为核验依据，围绕“训练、评分、评测或结果记录流程”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
训练、评分、评测或结果记录流程	依赖、配置和权限已满足	使用 src/railcap/__init__.py 对应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和重复执行一致性
训练、评分、评测或结果记录流程	缺少字段或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入半成品	检查错误码、日志脱敏和恢复动作

图示对应关系：本单元与“训练、评分、评测或结果记录流程”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.7 2.1 图像/视频输入、帧处理或识别结果管理——业务职责与边界

本单元以 src/railcap/benchmark.py 为核验依据，围绕“图像/视频输入、帧处理或识别结果管理”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 2

图像/视频输入、帧处理或识别结果管理



图示说明：从 src/railcap/benchmark.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-2 图像/视频输入、帧处理或识别结果管理详细处理图。图中从 src/railcap/benchmark.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/benchmark.py
处理过程	按状态推进，不跨越失败状态	图像/视频输入、帧处理或识别结果管理
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“图像/视频输入、帧处理或识别结果管理”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-2.svg，图示以 src/railcap/benchmark.py 为项目专属标识。

5.8 2.2 图像/视频输入、帧处理或识别结果管理——输入输出与数据约束

本单元以 src/railcap/benchmark.py 为核验依据，围绕“图像/视频输入、帧处理或识别结果管理”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/benchmark.py
处理过程	按状态推进，不跨越失败状态	图像/视频输入、帧处理或识别结果管理
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“图像/视频输入、帧处理或识别结果管理”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.9 2.3 图像/视频输入、帧处理或识别结果管理——状态转换与处理规则

本单元以 src/railcap/benchmark.py 为核验依据，围绕“图像/视频输入、帧处理或识别结果管理”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/benchmark.py
处理过程	按状态推进，不跨越失败状态	图像/视频输入、帧处理或识别结果管理
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“图像/视频输入、帧处理或识别结果管理”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.10 2.4 图像/视频输入、帧处理或识别结果管理——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":" 图 像 / 视 频 输 入 、 帧 处 理 或 识 别 结 果 管 理 ","payload":"*","page":1,"pageSize":10}; 成 功 响 应 :
{"code":0,"message":"ok","status":"completed","data":" 按 项 目 实 际 对 象 返 回 "}; 失 败 响 应 :
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/benchmark.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/benchmark.py
处理过程	按状态推进，不跨越失败状态	图像/视频输入、帧处理或识别结果管理
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“图像/视频输入、帧处理或识别结果管理”详细处理图一致，图中标示的输入、处
理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.11 2.5 图像/视频输入、帧处理或识别结果管理——异常分支与安全控制

本单元以 src/railcap/benchmark.py 为核验依据，围绕“图像/视频输入、帧处理或识别结果管理”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/benchmark.py
处理过程	按状态推进，不跨越失败状态	图像/视频输入、帧处理或识别结果管理
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“图像/视频输入、帧处理或识别结果管理”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.12 2.6 图像/视频输入、帧处理或识别结果管理——测试设计与验收准则

本单元以 src/railcap/benchmark.py 为核验依据，围绕“图像/视频输入、帧处理或识别结果管理”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
图像/视频输入、帧处理或识别结果管理	依赖、配置和权限已满足	使用 src/railcap/benchmark.py 对应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和重复执行一致性
图像/视频输入、帧处理或识别结果管理	缺少字段或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入半成品	检查错误码、日志脱敏和恢复动作

图示对应关系：本单元与“图像/视频输入、帧处理或识别结果管理”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.13 3.1 三维模型、地理场景或 WebGL 交互——业务职责与边界

本单元以 src/railcap/config.py 为核验依据，围绕“三维模型、地理场景或 WebGL 交互”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 3

三维模型、地理场景或 WebGL 交互



图示说明：从 src/railcap/config.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-3 三维模型、地理场景或 WebGL 交互详细处理图。图中从 src/railcap/config.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/config.py
处理过程	按状态推进，不跨越失败状态	三维模型、地理场景或 WebGL 交互
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“三维模型、地理场景或 WebGL 交互”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-3.svg，图示以 src/railcap/config.py 为项目专属标识。

5.14 3.2 三维模型、地理场景或 WebGL 交互——输入输出与数据约束

本单元以 src/railcap/config.py 为核验依据，围绕“三维模型、地理场景或 WebGL 交互”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/config.py
处理过程	按状态推进，不跨越失败状态	三维模型、地理场景或 WebGL 交互
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“三维模型、地理场景或 WebGL 交互”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.15 3.3 三维模型、地理场景或 WebGL 交互——状态转换与处理规则

本单元以 src/railcap/config.py 为核验依据，围绕“三维模型、地理场景或 WebGL 交互”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/config.py
处理过程	按状态推进，不跨越失败状态	三维模型、地理场景或 WebGL 交互
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“三维模型、地理场景或 WebGL 交互”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.16 3.4 三维模型、地理场景或 WebGL 交互——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":" 三 维 模 型 、 地 理 场 景 或 WebGL 交 互 ","payload":"*","page":1,"pageSize":10}; 成 功 响 应 :
{"code":0,"message":"ok","status":"completed","data":" 按 项 目 实 际 对 象 返 回 "}; 失 败 响 应 :
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/config.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/config.py
处理过程	按状态推进，不跨越失败状态	三维模型、地理场景或 WebGL 交互
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“三维模型、地理场景或 WebGL 交互”详细处理图一致，图中标示的输入、处理、
结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.17 3.5 三维模型、地理场景或 WebGL 交互——异常分支与安全控制

本单元以 src/railcap/config.py 为核验依据，围绕“三维模型、地理场景或 WebGL 交互”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/config.py
处理过程	按状态推进，不跨越失败状态	三维模型、地理场景或 WebGL 交互
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“三维模型、地理场景或 WebGL 交互”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.18 3.6 三维模型、地理场景或 WebGL 交互——测试设计与验收准则

本单元以 src/railcap/config.py 为核验依据，围绕“三维模型、地理场景或 WebGL 交互”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
三维模型、地理场景或 WebGL 交互	依赖、配置和权限已满足	使用 src/railcap/config.py 对应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和重复执行一致性
三维模型、地理场景或 WebGL 交互	缺少字段或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入半成品	检查错误码、日志脱敏和恢复动作

图示对应关系：本单元与“三维模型、地理场景或 WebGL 交互”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.19 4.1 知识库、向量检索或增强式问答链路——业务职责与边界

本单元以 src/railcap/evaluate.py 为核验依据，围绕“知识库、向量检索或增强式问答链路”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 4

知识库、向量检索或增强式问答链路



图示说明：从 src/railcap/evaluate.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-4 知识库、向量检索或增强式问答链路详细处理图。图中从 src/railcap/evaluate.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/evaluate.py
处理过程	按状态推进，不跨越失败状态	知识库、向量检索或增强式问答链路
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“知识库、向量检索或增强式问答链路”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-4.svg，图示以 src/railcap/evaluate.py 为项目专属标识。

5.20 4.2 知识库、向量检索或增强式问答链路——输入输出与数据约束

本单元以 src/railcap/evaluate.py 为核验依据，围绕“知识库、向量检索或增强式问答链路”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/evaluate.py
处理过程	按状态推进，不跨越失败状态	知识库、向量检索或增强式问答链路
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“知识库、向量检索或增强式问答链路”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.21 4.3 知识库、向量检索或增强式问答链路——状态转换与处理规则

本单元以 src/railcap/evaluate.py 为核验依据，围绕“知识库、向量检索或增强式问答链路”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/evaluate.py
处理过程	按状态推进，不跨越失败状态	知识库、向量检索或增强式问答链路
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“知识库、向量检索或增强式问答链路”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.22 4.4 知识库、向量检索或增强式问答链路——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":" 知 识 库 、 向 量 检 索 或 增 强 式 问 答 链 路 ","payload":"*","page":1,"pageSize":10}; 成 功 响 应 :
{"code":0,"message":"ok","status":"completed","data":" 按 项 目 实 际 对 象 返 回 "}; 失 败 响 应 :
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/evaluate.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/evaluate.py
处理过程	按状态推进，不跨越失败状态	知识库、向量检索或增强式问答链路
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“知识库、向量检索或增强式问答链路”详细处理图一致，图中标示的输入、处
理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.23 4.5 知识库、向量检索或增强式问答链路——异常分支与安全控制

本单元以 src/railcap/evaluate.py 为核验依据，围绕“知识库、向量检索或增强式问答链路”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/evaluate.py
处理过程	按状态推进，不跨越失败状态	知识库、向量检索或增强式问答链路
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“知识库、向量检索或增强式问答链路”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.24 4.6 知识库、向量检索或增强式问答链路——测试设计与验收准则

本单元以 src/railcap/evaluate.py 为核验依据，围绕“知识库、向量检索或增强式问答链路”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
知识库、向量检索或增强式问答链路	依赖、配置和权限已满足	使用 src/railcap/evaluate.py 对应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和重复执行一致性
知识库、向量检索或增强式问答链路	缺少字段或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入半成品	检查错误码、日志脱敏和恢复动作

图示对应关系：本单元与“知识库、向量检索或增强式问答链路”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.25 5.1 结果导出、格式转换或报告生成——业务职责与边界

本单元以 src/railcap/knowledge/__init__.py 为核验依据，围绕“结果导出、格式转换或报告生成”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 5

结果导出、格式转换或报告生成



图示说明：从 src/railcap/knowledge/__init__.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-5 结果导出、格式转换或报告生成详细处理图。图中从 src/railcap/knowledge/__init__.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/__init__.py
处理过程	按状态推进，不跨越失败状态	结果导出、格式转换或报告生成
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“结果导出、格式转换或报告生成”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-5.svg，图示以 src/railcap/knowledge/__init__.py 为项目专属标识。

5.26 5.2 结果导出、格式转换或报告生成——输入输出与数据约束

本单元以 src/railcap/knowledge/__init__.py 为核验依据，围绕“结果导出、格式转换或报告生成”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/__init__.py
处理过程	按状态推进，不跨越失败状态	结果导出、格式转换或报告生成
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“结果导出、格式转换或报告生成”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.27 5.3 结果导出、格式转换或报告生成——状态转换与处理规则

本单元以 src/railcap/knowledge/__init__.py 为核验依据，围绕“结果导出、格式转换或报告生成”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/__init__.py
处理过程	按状态推进，不跨越失败状态	结果导出、格式转换或报告生成
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“结果导出、格式转换或报告生成”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.28 5.4 结果导出、格式转换或报告生成——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":"结果导出、格式转换或报告生成","payload":"*","page":1,"pageSize":10}; 成功响应：
{"code":0,"message":"ok","status":"completed","data":"按项目实际对象返回"}; 失败响应：
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/knowledge/__init__.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/__init__.py
处理过程	按状态推进，不跨越失败状态	结果导出、格式转换或报告生成
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“结果导出、格式转换或报告生成”详细处理图一致，图中标示的输入、处理、结
果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.29 5.5 结果导出、格式转换或报告生成——异常分支与安全控制

本单元以 src/railcap/knowledge/__init__.py 为核验依据，围绕“结果导出、格式转换或报告生成”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/__init__.py
处理过程	按状态推进，不跨越失败状态	结果导出、格式转换或报告生成
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“结果导出、格式转换或报告生成”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.30 5.6 结果导出、格式转换或报告生成——测试设计与验收准则

本单元以 src/railcap/knowledge/__init__.py 为核验依据，围绕“结果导出、格式转换或报告生成”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
结果导出、格式转换或报告生成	依赖、配置和权限已满足	使用 src/railcap/knowledge/__init__.py 对应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和重复执行一致性
结果导出、格式转换或报告生成	缺少字段或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入半成品	检查错误码、日志脱敏和恢复动作

图示对应关系：本单元与“结果导出、格式转换或报告生成”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.31 6.1 规则判断、风险识别或异常分支——业务职责与边界

本单元以 src/railcap/knowledge/constraints.py 为核验依据，围绕“规则判断、风险识别或异常分支”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 6

规则判断、风险识别或异常分支



图示说明：从 src/railcap/knowledge/constraints.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-6 规则判断、风险识别或异常分支详细处理图。图中从 src/railcap/knowledge/constraints.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/constraints.py
处理过程	按状态推进，不跨越失败状态	规则判断、风险识别或异常分支
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“规则判断、风险识别或异常分支”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-6.svg，图示以 src/railcap/knowledge/constraints.py 为项目专属标识。

5.32 6.2 规则判断、风险识别或异常分支——输入输出与数据约束

本单元以 src/railcap/knowledge/constraints.py 为核验依据，围绕“规则判断、风险识别或异常分支”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/constraints.py
处理过程	按状态推进，不跨越失败状态	规则判断、风险识别或异常分支
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“规则判断、风险识别或异常分支”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.33 6.3 规则判断、风险识别或异常分支——状态转换与处理规则

本单元以 src/railcap/knowledge/constraints.py 为核验依据，围绕“规则判断、风险识别或异常分支”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/constraints.py
处理过程	按状态推进，不跨越失败状态	规则判断、风险识别或异常分支
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“规则判断、风险识别或异常分支”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.34 6.4 规则判断、风险识别或异常分支——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":" 规则 判 断 、 风 险 识 别 或 异 常 分 支 ","payload":"*","page":1,"pageSize":10}; 成 功 响 应 :
{"code":0,"message":"ok","status":"completed","data":" 按 项 目 实 际 对 象 返 回 "}; 失 败 响 应 :
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/knowledge/constraints.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/constraints.py
处理过程	按状态推进，不跨越失败状态	规则判断、风险识别或异常分支
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“规则判断、风险识别或异常分支”详细处理图一致，图中标示的输入、处理、结
果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.35 6.5 规则判断、风险识别或异常分支——异常分支与安全控制

本单元以 `src/railcap/knowledge/constraints.py` 为核验依据，围绕“规则判断、风险识别或异常分支”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	<code>src/railcap/knowledge/constraints.py</code>
处理过程	按状态推进，不跨越失败状态	规则判断、风险识别或异常分支
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“规则判断、风险识别或异常分支”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.36 6.6 规则判断、风险识别或异常分支——测试设计与验收准则

本单元以 src/railcap/knowledge/constraints.py 为核验依据，围绕“规则判断、风险识别或异常分支”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
规则判断、 风险识别或 异常分支	依赖、配置和权限 已满足	使用 src/railcap/knowledge/constraints.py 对 应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和 重复执行一致性
规则判断、 风险识别或 异常分支	缺少字段 或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入 半成品	检查错误码、 日志脱敏和恢复动作

图示对应关系：本单元与“规则判断、风险识别或异常分支”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.37 7.1 代码入口 align 对应的可核验处理单元——业务职责与边界

本单元以 src/railcap/knowledge/ontology.py 为核验依据，围绕“代码入口 align 对应的可核验处理单元”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 7

代码入口 align 对应的可核验处理单元



图示说明：从 src/railcap/knowledge/ontology.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-7 代码入口 align 对应的可核验处理单元详细处理图。图中从 src/railcap/knowledge/ontology.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/ontology.py
处理过程	按状态推进，不跨越失败状态	代码入口 align 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 align 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-7.svg，图示以 src/railcap/knowledge/ontology.py 为项目专属标识。

5.38 7.2 代码入口 align 对应的可核验处理单元——输入输出与数据约束

本单元以 src/railcap/knowledge/ontology.py 为核验依据，围绕“代码入口 align 对应的可核验处理单元”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/ontology.py
处理过程	按状态推进，不跨越失败状态	代码入口 align 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 align 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.39 7.3 代码入口 align 对应的可核验处理单元——状态转换与处理规则

本单元以 src/railcap/knowledge/ontology.py 为核验依据，围绕“代码入口 align 对应的可核验处理单元”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/ontology.py
处理过程	按状态推进，不跨越失败状态	代码入口 align 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 align 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.40 7.4 代码入口 align 对应的可核验处理单元——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":" 代 码 入 口 align 对 应 的 可 核 验 处 理 单 元 ","payload":"*","page":1,"pageSize":10}; 成 功 响 应 :
{"code":0,"message":"ok","status":"completed","data":" 按 项 目 实 际 对 象 返 回 "}; 失 败 响 应 :
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/knowledge/ontology.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/ontology.py
处理过程	按状态推进，不跨越失败状态	代码入口 align 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 align 对应的可核验处理单元”详细处理图一致，图中标示的输入、处
理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.41 7.5 代码入口 align 对应的可核验处理单元——异常分支与安全控制

本单元以 src/railcap/knowledge/ontology.py 为核验依据，围绕“代码入口 align 对应的可核验处理单元”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/knowledge/ontology.py
处理过程	按状态推进，不跨越失败状态	代码入口 align 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 align 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.42 7.6 代码入口 align 对应的可核验处理单元——测试设计与验收准则

本单元以 src/railcap/knowledge/ontology.py 为核验依据，围绕“代码入口 align 对应的可核验处理单元”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
代码入口 align 对应的可核验处理单元	依赖、配置和权限已满足	使用 src/railcap/knowledge/ontology.py 对应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和重复执行一致性
代码入口 align 对应的可核验处理单元	缺少字段或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入半成品	检查错误码、日志脱敏和恢复动作

图示对应关系：本单元与“代码入口 align 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.43 8.1 代码入口 center 对应的可核验处理单元——业务职责与边界

本单元以 src/railcap/metrics/__init__.py 为核验依据，围绕“代码入口 center 对应的可核验处理单元”建立可追踪设计。说明该设计单元解决的业务问题、责任边界、前置条件、完成条件及与相邻模块的衔接。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

资源受限铁道知识图片描述软件 - 详细处理单元 8

代码入口 center 对应的可核验处理单元



图示说明：从 src/railcap/metrics/__init__.py 进入，经过输入边界、核心机制、结果状态和异常恢复的闭环；节点均对应本项目核验到的代码或配置边界。

图 5-8 代码入口 center 对应的可核验处理单元详细处理图。图中从 src/railcap/metrics/__init__.py 进入，经过参数校验、核心处理、状态结果和异常恢复，节点与本节逻辑逐一对应。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/metrics/__init__.py
处理过程	按状态推进，不跨越失败状态	代码入口 center 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 center 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；设计图文件为 images/diagrams/module-8.svg，图示以 src/railcap/metrics/__init__.py 为项目专属标识。

5.44 8.2 代码入口 center 对应的可核验处理单元——输入输出与数据约束

本单元以 src/railcap/metrics/__init__.py 为核验依据，围绕“代码入口 center 对应的可核验处理单元”建立可追踪设计。说明输入来源、字段或文件约束、校验顺序、输出对象、持久化边界和空结果处理。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/metrics/__init__.py
处理过程	按状态推进，不跨越失败状态	代码入口 center 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 center 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.45 8.3 代码入口 center 对应的可核验处理单元——状态转换与处理规则

本单元以 src/railcap/metrics/__init__.py 为核验依据，围绕“代码入口 center 对应的可核验处理单元”建立可追踪设计。说明正常状态、处理中状态、成功状态、失败状态和重试/恢复条件，避免把异常状态当成成功结果。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/metrics/__init__.py
处理过程	按状态推进，不跨越失败状态	代码入口 center 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 center 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.46 8.4 代码入口 center 对应的可核验处理单元——公共报文与接口约定

公共报文采用“请求元数据—业务输入—处理状态—结果或错误”的稳定结构。请求示例：
{"requestId":"*","operation":" 代 码 入 口 center 对 应 的 可 核 验 处 理 单 元 ","payload":"*","page":1,"pageSize":10}; 成 功 响 应 :
{"code":0,"message":"ok","status":"completed","data":" 按 项 目 实 际 对 象 返 回 "}; 失 败 响 应 :
{"code":"INPUT_INVALID","message":"请检查输入条件","details":"***"}。字段名仅用于表达接口语义，
真实调用仍以 src/railcap/metrics/__init__.py 的函数签名、路由或命令参数为准。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、
权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/metrics/__init__.py
处理过程	按状态推进，不跨越失败状态	代码入口 center 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 center 对应的可核验处理单元”详细处理图一致，图中标示的输入、处
理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.47 8.5 代码入口 center 对应的可核验处理单元——异常分支与安全控制

本单元以 src/railcap/metrics/__init__.py 为核验依据，围绕“代码入口 center 对应的可核验处理单元”建立可追踪设计。说明参数缺失、格式错误、依赖不可用、权限不足、资源耗尽、重复提交和日志审计等处理策略。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

异常优先在边界处拦截，返回可识别的错误类型并记录必要上下文；不得继续写入不完整结果。对可恢复错误采用重试、重新加载、回滚或人工复核，对不可恢复错误保留日志和输入摘要，避免输出敏感内容。

设计项目	当前设计要求	代码/配置核验方式
输入边界	先校验后处理，拒绝不完整数据	src/railcap/metrics/__init__.py
处理过程	按状态推进，不跨越失败状态	代码入口 center 对应的可核验处理单元
结果边界	结果可读取、可追踪、可恢复	返回对象、文件、日志或数据表

图示对应关系：本单元与“代码入口 center 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。

5.48 8.6 代码入口 center 对应的可核验处理单元——测试设计与验收准则

本单元以 src/railcap/metrics/__init__.py 为核验依据，围绕“代码入口 center 对应的可核验处理单元”建立可追踪设计。说明前置条件、输入数据、操作步骤、预期结果、验证要点和不通过时的回退或补证据方式。处理时先确认前置条件，再按输入校验、核心逻辑、状态更新和结果输出顺序执行；所有状态应能通过返回对象、日志、文件、数据库记录或页面状态回读验证。

设计要求异常路径与正常路径同样可验证：输入为空、类型不符、文件不可读、外部服务超时、资源不足、权限不足及重复操作分别给出受控结果，恢复后不能产生重复数据或状态倒退。

测试对象	前置条件	输入/步骤	预期结果	验证要点
代码入口 center 对应的可核验处理单元	依赖、配置和权限已满足	使用 src/railcap/metrics/__init__.py 对应入口提交有效输入	得到稳定结果并记录状态	检查结果、日志、持久化和重复执行一致性
代码入口 center 对应的可核验处理单元	缺少字段或依赖不可用	提交空值、错误格式或模拟不可用依赖	返回明确错误，不写入半成品	检查错误码、日志脱敏和恢复动作

图示对应关系：本单元与“代码入口 center 对应的可核验处理单元”详细处理图一致，图中标示的输入、处理、结果和异常节点分别对应本节的设计约束；本节沿用同一机制图作为上下文索引，不重复绘制相同逻辑。